Monitorowanie zasobów i procesów w Ubuntu Server

CPU, RAM, dysk, procesy, logi

Systemy operacyjne - klasa 2

Ubuntu Server

Część 1: CPU, RAM, Dysk i zarządzanie procesami

Cel lekcji

Zrozumienie, jak system operacyjny zarządza zasobami oraz jak administrator może monitorować ich zużycie w czasie rzeczywistym.

Po tej lekcji uczeń powinien rozumieć

- czym różni się program od procesu oraz czym są demony
- jak interpretować obciążenie CPU (load average)
- jak sprawdzić zajętość i strukturę pamięci RAM oraz przestrzeni wymiany (SWAP)
- jak sprawdzić zajętość dysku (w tym inody)
- do czego służą narzędzia ps, top i htop

Program a proces - pogłębienie

- **Program:** plik wykonywalny (lub skrypt) zapisany na dysku twardym (np. `/bin/bash`, `/usr/sbin/nginx`).
- **Proces:** uruchomiona instancja programu załadowana do pamięci RAM, wykonująca instrukcje procesora.
- Jeden program może uruchomić wiele procesów (np. przeglądarka internetowa uruchamia osobny proces dla każdej karty).
- **Demony (daemons):** procesy działające w tle, bez interakcji z użytkownikiem (np. serwer WWW, serwer SSH). Zazwyczaj kończą się na literę 'd' (np. `sshd`, `systemd`).

Identyfikacja procesów

- Każdy proces otrzymuje unikalny numer identyfikacyjny – **PID** (Process ID).
- Proces o PID 1 to proces inicjujący (w nowszych systemach zazwyczaj `systemd`), który uruchamia wszystkie pozostałe.
- Procesy tworzą strukturę drzewa – proces nadrzędny (rodzic - PPID) uruchamia procesy podrzędne (dzieci).

Dlaczego monitorujemy serwer?

- **Podejście proaktywne:** wykrywanie anomalii zanim staną się awarią (np. dysk zapełnia się w tempie 1GB dziennie – mamy tydzień na reakcję).
- **Podejście reaktywne:** diagnoza, gdy usługa już przestała działać (np. strona wolno się ładuje z powodu braku RAMu).
- Brak miejsca na dysku potrafi zatrzymać usługi (np. baza danych nie może zapisać nowych rekordów ani logów, więc ulega awarii).
- Diagnostyka opiera się na twardych danych i pomiarach, a nie na zgadywaniu.

Zarządzanie procesami - top i htop

Narzędzia te działają w czasie rzeczywistym.

```
top
htop (wersja z bardziej przyjaznym interfejsem graficznym,
często wymaga doinstalowania: apt install htop)
```

Co oznaczają parametry w top?

- **load average:** 3 liczby oznaczające średnie obciążenie kolejki procesora w ciągu 1, 5 i 15 minut. Wartość 1.0 na 1-rdzeniowym procesorze oznacza 100% utylizacji.
- **%CPU:** procent czasu procesora zużywanego przez dany proces.
- **%MEM:** procent używanej pamięci RAM fizycznej.

Migawka procesów - ps

Polecenie ps wyświetla stan procesów w danej chwili (jak zdjęcie).

```
ps aux
```

Rozwinięcie flag:

- **a** - pokazuje procesy wszystkich użytkowników.
- **u** - wyświetla szczegółowe informacje (użytkownik, CPU, MEM).
- **x** - pokazuje również procesy odłączone od terminala (np. demony).

Przydatne: wyszukiwanie konkretnego procesu potokiem:
`ps aux | grep nginx`

Pamięć RAM i SWAP

Pamięć podręczna i przestrzeń wymiany.

```
free -h
cat /proc/meminfo
```

- Flaga -**h** (human-readable) formatuje wynik na MB/GB zamiast bajtów.
- **SWAP** to przestrzeń wymiany na dysku twardym. Jeśli brakuje RAM-u, system zrzuca najrzadziej używane dane na dysk.
- Wysokie zużycie SWAP oznacza drastyczny spadek wydajności (dysk jest wolniejszy od RAM-u).

Dysk - przestrzeń i inody

```
df -h
df -i
du -sh /var/log
lsblk
```

- **df (disk free)** pokazuje wolne miejsce na zamontowanych partycjach.
- **df -i** pokazuje zużycie **inodów** (i-węzłów). Możesz mieć gigabajty wolnego miejsca, ale jeśli masz miliony maleńkich plików, wyczerpiesz inody i system nie pozwoli zapisać nowego pliku!
- **du (disk usage)** liczy rozmiar plików w danym katalogu (flaga **-s** podsumowuje wynik, żeby nie listować każdego pliku z osobna).
- **lsblk** wyświetla strukturę fizycznych dysków i ich partycji.

Przerwa między lekcjami



Ciąg dalszy (logi, systemctl, rozwiązywanie problemów) na następnej lekcji!

Część 2: Logi, usługi i rozwiązywanie problemów

Cel lekcji

Nauka lokalizowania i analizy logów systemowych, kontrolowania usług oraz prawidłowego diagnozowania problemów.

Katalog /var i jego znaczenie

- `/var` (variable) to miejsce, gdzie system przechowuje dane, które ulegają częstym zmianom.
- Znajdują się tu logi (`/var/log`), bazy danych (np. `/var/lib/mysql`), czy pliki stron WWW (`/var/www`).
- Niekontrolowany przyrost logów (np. wywołany błędem aplikacji spamującym komunikaty) potrafi błyskawicznie zapełnić ten katalog.
- Pełny `/var` to gwarantowany problem z działaniem serwera!

Wprowadzenie do logów

- **Log** to wpis w dzienniku zdarzeń systemu lub konkretnej usługi. Zawiera zazwyczaj czas, nazwę usługi, rodzaj komunikatu (INFO, WARNING, ERROR) i sam komunikat.
- Ważne pliki w tradycyjnym systemie:
  - `/var/log/syslog` - główny log systemowy Ubuntu.
  - `/var/log/auth.log` - logi logowania i autoryzacji (np. próby SSH).
  - `/var/log/dmesg` - komunikaty jądra i sprzętu (np. podpięcie nowego dysku).
- Obecnie głównym narzędziem do logów w systemach z *systemd* jest **journalctl**.

Narzędzie journalctl

```
journalctl                  # wyświetla wszystko od początku
journalctl -xe              # skacze na koniec (-e) i podaje detale (-x)
journalctl -u ssh           # wyświetla tylko logi konkretnej usługi (-u)
journalctl --since "today" # filtruje logi według czasu
journalctl -f               # śledzi logi na bieżąco (follow) - bardzo przydatne!
journalctl -p err           # pokazuje tylko błędy (priority error)
```

Stan usług - systemctl status

```
systemctl status nazwa_uslugi
(np. systemctl status apache2)
```

To polecenie to pierwszy krok po zgłoszeniu awarii. Sprawdzamy:

- **Active:** czy świeci się na zielono (active/running) czy na czerwono (failed).
- Od kiedy usługa działa (uptime usługi).
- Czy usługa jest włączona na start systemu (**enabled**/**disabled**).
- Na samym dole wydruku pojawi się klika ostatnich linijek logów z tej usługi.

Kończenie procesów - polecenie kill

- `kill` wysyła tzw. **sygnał** do procesu w oparciu o jego PID (który znajdujemy np. poleceniem `pidof nginx` lub `ps aux`).
- **kill PID (sygnał 15 - SIGTERM):** Uprzejma prośba o zakończenie pracy. Program ma czas zapisać dane, zamknąć pliki i kulturalnie się wyłączyć. Tego używamy domyślnie!
- **kill -9 PID (sygnał 9 - SIGKILL):** Brute force. System zabija proces natychmiastowo. Używamy **tylko** gdy proces zawiśnie i nie reaguje na SIGTERM. Może skutkować uszkodzeniem zapisywanych danych.

Przykład metodycznej diagnozy

Zgłoszenie: *"Strona szkoły nie ładuje się!"*

1. **Warstwa sieci:** Czy serwer w ogóle żyje? (`ping adres_IP`).
2. **Zasoby bieżące:** Czy serwer nie jest przeciążony? (`top`, `free -h`).
3. **Stan dysku:** Czy jest miejsce na dysku? (`df -h`).
4. **Stan usługi:** Czy Apache/Nginx działa? (`systemctl status apache2`).
5. **Nasłuch:** Czy usługa faktycznie słucha na porcie 80/443? (`ss -tulpn`).
6. **Logi (Ostateczna prawda):** Czego brakuje do uruchomienia? (`journalctl -xe -u apache2` lub `tail /var/log/apache2/error.log`).

Notatka do zeszytu

- Użycie procesora (load average, %CPU) i RAM (`free -h`, zjawisko SWAPowania) monitorujemy narzędziami `top`/`htop`.
- Brak miejsca na dysku (sprawdzamy `df -h`) to najczęstsza, banalna przyczyna powstawania poważnych awarii usług.
- Procesy wyłączamy poleceniem `kill PID` (domyślnie SIGTERM, ostatecznie SIGKILL -9).
- Logi, czytane za pomocą `journalctl`, są kluczem do precyzyjnego diagnozowania problemów.

Mini-pytania kontrolne na koniec

1. Co oznacza wysokie zużycie pamięci SWAP?
2. Czym różni się parametr df -h od df -i?
3. Jaką komendą na bieżąco śledzisz dodawanie nowych logów?
4. Dlaczego komendy `kill -9` używamy tylko w ostateczności?
5. Jak szybko sprawdzić, dlaczego `systemctl restart usługa` zakończyło się błędem?